

Escalabilidade do Pipeline B3

Passo a passo para evoluir o projeto de 4 para 500 tickers (ou o pregão completo) — documento de referência técnica, não implementado nesta versão do projeto.

Contexto

O pipeline atual (KNIME → Databricks, arquitetura Medallion) foi construído e validado com 4 tickers (PETR4, VALE3, MGLU3, ITUB4), escolhidos por viabilidade dentro do prazo do projeto. A arquitetura já foi desenhada com escala em mente (Widgets parametrizáveis, MERGE idempotente por chave, Volumes particionados por data), mas alguns pontos exigem revisão explícita antes de comportar 500 tickers ou o pregão completo da B3. Este documento resume o que precisaria mudar, em que ordem, e os riscos de cada etapa.

Passo 1 — Fonte dinâmica de tickers

Hoje a lista de tickers é um dropdown fixo no Widget do notebook de ingestão (4 opções codificadas no código). Isso não escala.

- Criar uma tabela de referência (**landing.tickers_referencia** ou similar) com a lista de tickers ativos, em vez de lista fixa no código.
- O notebook de ingestão passa a ler essa tabela para montar a lista de chamadas à API, em vez de depender do Widget dropdown.
- O Widget de override manual (rodar 1 ticker específico) pode ser mantido, mas como filtro sobre essa tabela, não como lista fixa.

Passo 2 — Paginação e limites da API

A API brapi.dev, usada em modo sandbox gratuito, tem limites de requisição não confirmados para volume alto. Antes de escalar, é necessário:

- Validar o rate limit real do plano gratuito (requisições por minuto/hora) com testes controlados, não assumir.
- Se o plano gratuito não suportar 500 chamadas em sequência, implementar requisições em lote (batch), com pausas controladas entre grupos de chamadas.
- Avaliar se a API oferece endpoint de cotação em lote (múltiplos tickers por chamada), o que reduziria drasticamente o número de requisições necessárias.

Passo 3 — Volume de dados por camada

Com 500 tickers, o volume de dados por execução cresce ~125x em relação ao atual. Pontos de atenção por camada:

Camada	Impacto do aumento de escala
Landing	Arquivo JSON por dia cresce proporcionalmente — sem impacto estrutural, só volume.
Bronze	MERGE por chave (ticker + data_referencia) continua válido; volume maior pode exigir otimização de particion
Silver	Deduplicação e quarentena continuam válidas sem redesenho; volume de quarentena pode crescer proporcion

Gold	Índice acumulado (capitalização composta) reprocessa o histórico completo a cada execução (ver ADR-09) —
------	--

Passo 4 — Quando considerar Programação Orientada a Objetos

Hoje o projeto não usa OOP, deliberadamente — uma única fonte de dados não justifica a abstração. Esse cálculo muda em dois cenários de escala:

- **Múltiplas fontes de dados** (ex.: adicionar câmbio, outro provedor de cotação, ou um segundo índice): uma classe base de ingestão com subclasses por fonte passaria a valer o investimento.
- **Lógica de negócio divergente por grupo de tickers** (ex.: tratamento diferente para ações vs. FIIs vs. BDRs): justificaria uma hierarquia de classes de processamento, não apenas de ingestão.
- Com 500 tickers da mesma fonte e mesma lógica, OOP ainda não seria estritamente necessário — o ganho viria mais de paralelização/particionamento do que de abstração de classes.

Passo 5 — Orquestração e Job

- O Job atual (5 Tasks lineares) continua válido estruturalmente — não precisa de redesenho de dependências.
- Tempo de execução por Task deve ser medido em escala real antes de assumir que o agendamento às 17h15 continua com margem suficiente até o fechamento.
- Compute Serverless (Free Edition) tem limites de concorrência — validar se o volume maior de dados não esbarra em algum teto de recursos do plano gratuito.

Ordem recomendada de execução

Sequência sugerida para minimizar risco, testando cada mudança isoladamente antes de acumular a próxima:

1. Validar rate limit real da API com um teste controlado (ex.: 50 tickers).
2. Criar a tabela de referência de tickers e migrar o notebook de ingestão para lê-la.
3. Rodar o pipeline completo com um grupo intermediário (ex.: 50 tickers) e medir tempo/volume por camada antes de saltar para 500.
4. Revisar a estratégia do índice acumulado (Passo 3) para cálculo incremental, se o reproprocessamento completo do histórico se tornar lento na escala intermediária.
5. Só então escalar para 500 tickers ou o pregão completo, validando novamente tempo de execução e limites do compute Serverless.

Este é um documento de planejamento técnico, não uma implementação — nenhuma das etapas acima foi executada nesta versão do projeto. O objetivo é registrar o raciocínio de arquitetura para uma eventual continuidade, sem comprometer o escopo e o prazo da entrega atual.